

M Extensions

MO

HALBORN

M Extensions - MO

Prepared by:  HALBORN

Last Updated 09/19/2025

Date of Engagement: August 11th, 2025 - September 1st, 2025

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED



TABLE OF CONTENTS

1. Introduction
2. Assessment summary
3. Test approach and methodology
4. Risk methodology
5. Scope
6. Assessment summary & findings overview
7. Findings & Tech Details
7.1 Inboxitem release instruction is not enforced
7.2 Insufficient m-token mint validation during earn program initialization
7.3 Risk of yield over-distribution
7.4 Token 2022 program is not enforced
7.5 Missing field in initializer of extglobalv2 account
7.6 Risk of dos due to insufficient accounts validation
7.7 Risk of unhandled index out of bounds error
8. Automated Testing

1. Introduction

M0 team engaged Halborn to conduct a security assessment on their earn, portal, m_ext and ext_swap Solana programs beginning on August 11, 2025, and ending on September 1, 2025. The security assessment was scoped to the Solana Programs provided in solana-m and solana-m-extensions GitHub repositories. Commit hashes and further details can be found in the Scope section of this report.

The Solana M platform is a system for managing and distributing yield to token holders through multiple coordinated programs. The purpose of the system is to allow bridging M token to Solana and maintaining the yield earning features found on EVM chains.

The scope of this assessment was to review design changes that moved the **M-token yield distribution fully on-chain**, replacing the previous implementation that relied on an **off-chain yield crank**.

This report includes only **new findings** and does not repeat findings from the prior assessment that were acknowledged by the M0 team but left unchanged in the implementation.

The M0 team also provided updates to the code base with additional fixes and introduced a new feature that allows disabling yield distribution in the crank variant. The corresponding pull requests are listed in the *Scope* section.

2. Assessment Summary

Halborn was provided 3 weeks for the engagement and assigned one full-time security engineer to review the security of the Solana Programs in scope. The engineer is a blockchain and smart contract security expert with advanced smart contract hacking skills, and deep knowledge of multiple blockchain protocols.

The purpose of the assessment is to:

- Identify potential security issues within the Solana Programs.
- Ensure that smart contract functionality operates as intended.
- Ensure that the migration to new program and data accounts versions operates as intended.

In summary, **Halborn** identified some improvements to reduce the likelihood and impact of risks, which were mostly addressed by the **M0 team**. The main ones were the following:

- **Ensure each InboxItem is released using the appropriate instruction.**
- **Perform full validation of the new M-token mint during both migration and new deployments.**
- **Ensure that the last m index is fully synchronized before re-adding the extension as an earner and syncing the extension index.**

3. Test Approach And Methodology

Halborn performed a combination of a manual review of the source code and automated security testing to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of the program assessment. While manual testing is recommended to uncover flaws in business logic, processes, and implementation; automated testing techniques help enhance coverage of programs and can quickly identify items that do not follow security best practices.

The following phases and associated tools were used throughout the term of the assessment:

- Research into the architecture, purpose, and use of the platform.
- Manual program source code review to identify business logic issues.
- Mapping out possible attack vectors
- Thorough assessment of safety and usage of critical Rust variables and functions in scope that could lead to arithmetic vulnerabilities.
- Scanning dependencies for known vulnerabilities (**cargo audit**).
- Local runtime testing (**solana-test-framework**)

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker’s control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (<i>C</i>)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (<i>r</i>)	None (R:N) Partial (R:P) Full (R:F)	1 0.5 0.25
Scope (<i>s</i>)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4

SEVERITY	SCORE VALUE RANGE
Informational	0 - 1.9

5. SCOPE

REPOSITORIES

(a) Repository: [solana-m](#)

(b) Assessed Commit ID: <https://github.com/m0-foundation/solana-m/pull/166>

(c) Items in scope:

- programs/earn/src/instructions/admin/initialize.rs
- programs/earn/src/instructions/open/add_registrar_earner.rs
- programs/earn/src/instructions/portal/propagate_index.rs
- programs/earn/src/utils/conversion.rs
- programs/portal/src/error.rs
- programs/portal/src/instructions/admin.rs
- programs/portal/src/instructions/redeem.rs
- programs/portal/src/instructions/release_inbound.rs
- programs/portal/src/instructions/release_inbound_extension.rs
- programs/portal/src/instructions/resolve_execute.rs
- programs/portal/src/instructions/transfer.rs
- programs/portal/src/instructions/transfer_extension.rs
- programs/portal/src/lib.rs

Out-of-Scope: Third party dependencies and economic attacks.

(a) Repository: [solana-m-extensions](#)

(b) Assessed Commit ID: <https://github.com/m0-foundation/solana-m-extensions/pull/59>

(c) Items in scope:

- programs/m_ext/src/errors.rs
- programs/m_ext/src/instructions/claim_fees.rs
- programs/m_ext/src/instructions/crank/admin/add_earn_manager.rs
- programs/m_ext/src/instructions/crank/earn_authority/claim_for.rs
- programs/m_ext/src/instructions/crank/earn_authority/sync.rs
- programs/m_ext/src/instructions/crank/earn_manager/add_earner.rs
- programs/m_ext/src/instructions/crank/earn_manager/configure.rs
- programs/m_ext/src/instructions/crank/earner/set_recipient.rs
- programs/m_ext/src/instructions/crank/remove_orphaned_earner.rs
- programs/m_ext/src/instructions/initialize.rs
- programs/m_ext/src/instructions/migrate.rs
- programs/m_ext/src/instructions/scaled_ui/set_fee.rs

- programs/m_ext/src/instructions/scaled_ui/sync.rs
- programs/m_ext/src/instructions/transfer_admin.rs
- programs/m_ext/src/instructions/unwrap.rs
- programs/m_ext/src/instructions/wrap.rs
- programs/m_ext/src/state/mod.rs
- programs/m_ext/src/utils/conversion.rs

Out-of-Scope: Third party dependencies and economic attacks.

(a) Repository: [solana-m-extensions](#)

(b) Assessed Commit ID: <https://github.com/m0-foundation/solana-m-extensions/pull/38/commits/c5b7833b9935a4311aa201a9e98a12299bfc1558>

(c) Items in scope:

- Anchor.toml
- Cargo.toml
- programs/ext_swap/Cargo.toml
- programs/ext_swap/src/errors.rs
- programs/ext_swap/src/instructions/initialize.rs
- programs/ext_swap/src/instructions/swap.rs
- programs/ext_swap/src/instructions/unwrap.rs
- programs/ext_swap/src/instructions/whitelist.rs
- programs/ext_swap/src/instructions/wrap.rs
- programs/ext_swap/src/lib.rs
- programs/ext_swap/src/state.rs
- programs/m_ext/Cargo.toml
- programs/m_ext/src/errors.rs
- programs/m_ext/src/instructions/claim_fees.rs
- programs/m_ext/src/instructions/crank/admin/add_earn_manager.rs
- programs/m_ext/src/instructions/crank/admin/mod.rs
- programs/m_ext/src/instructions/crank/admin/remove_earn_manager.rs
- programs/m_ext/src/instructions/crank/admin/set_earn_authority.rs
- programs/m_ext/src/instructions/crank/earn_authority/claim_for.rs
- programs/m_ext/src/instructions/crank/earn_authority/mod.rs
- programs/m_ext/src/instructions/crank/earn_authority/sync.rs
- programs/m_ext/src/instructions/crank/earn_manager/add_earner.rs
- programs/m_ext/src/instructions/crank/earn_manager/configure.rs
- programs/m_ext/src/instructions/crank/earn_manager/mod.rs
- programs/m_ext/src/instructions/crank/earn_manager/remove_earner.rs
- programs/m_ext/src/instructions/crank/earn_manager/transfer_earner.rs
- programs/m_ext/src/instructions/crank/earner/mod.rs
- programs/m_ext/src/instructions/crank/earner/set_recipient.rs

- programs/m_ext/src/instructions/crank/mod.rs
- programs/m_ext/src/instructions/crank/remove_orphaned_earner.rs
- programs/m_ext/src/instructions/initialize.rs
- programs/m_ext/src/instructions/manage_wrap_authority.rs
- programs/m_ext/src/instructions/migrate.rs
- programs/m_ext/src/instructions/mod.rs
- programs/m_ext/src/instructions/scaled_ui/mod.rs
- programs/m_ext/src/instructions/scaled_ui/set_fee.rs
- programs/m_ext/src/instructions/scaled_ui/sync.rs
- programs/m_ext/src/instructions/set_fee.rs
- programs/m_ext/src/instructions/set_m_mint.rs
- programs/m_ext/src/instructions/sync.rs
- programs/m_ext/src/instructions/transfer_admin.rs
- programs/m_ext/src/instructions/unwrap.rs
- programs/m_ext/src/instructions/wrap.rs
- programs/m_ext/src/lib.rs
- programs/m_ext/src/state/earn_manager.rs
- programs/m_ext/src/state/earner.rs
- programs/m_ext/src/state/mod.rs
- programs/m_ext/src/utis/conversion.rs
- programs/m_ext/src/utis/token.rs

Out-of-Scope: Third party dependencies and economic attacks.

(a) Repository: [solana-m](#)

(b) Assessed Commit ID: <https://github.com/m0-foundation/solana-m/pull/138/commits/ea6263519d588e9267c0a6c1e0b33ee6c95e0a61>

(c) Items in scope:

- Anchor.toml
- Cargo.toml
- programs/earn/Cargo.toml
- programs/earn/src/constants.rs
- programs/earn/src/errors.rs
- programs/earn/src/instructions/admin/initialize.rs
- programs/earn/src/instructions/admin/mod.rs
- programs/earn/src/instructions/admin/recover.rs
- programs/earn/src/instructions/admin/set_claim_cooldown.rs
- programs/earn/src/instructions/admin/set_earn_authority.rs
- programs/earn/src/instructions/earn_authority/claim_for.rs
- programs/earn/src/instructions/earn_authority/complete_claims.rs
- programs/earn/src/instructions/earn_authority/mod.rs

- programs/earn/src/instructions/mod.rs
- programs/earn/src/instructions/open/add_registrar_earner.rs
- programs/earn/src/instructions/open/mod.rs
- programs/earn/src/instructions/open/remove_registrar_earner.rs
- programs/earn/src/instructions/portal/mod.rs
- programs/earn/src/instructions/portal/propagate_index.rs
- programs/earn/src/lib.rs
- programs/earn/src/state.rs
- programs/earn/src/state/earner.rs
- programs/earn/src/state/global.rs
- programs/earn/src/state/mod.rs
- programs/earn/src/utils/conversion.rs
- programs/earn/src/utils/merkle_proof.rs
- programs/earn/src/utils/mod.rs
- programs/earn/src/utils/token.rs
- programs/portal/Cargo.toml
- programs/portal/src/bitmap.rs
- programs/portal/src/config.rs
- programs/portal/src/error.rs
- programs/portal/src/instructions/admin.rs
- programs/portal/src/instructions/initialize.rs
- programs/portal/src/instructions/luts.rs
- programs/portal/src/instructions/mark_outbox_item_as_released.rs
- programs/portal/src/instructions/mod.rs
- programs/portal/src/instructions/redeem.rs
- programs/portal/src/instructions/release_inbound.rs
- programs/portal/src/instructions/release_inbound_extension.rs
- programs/portal/src/instructions/resolve_execute.rs
- programs/portal/src/instructions/transfer.rs
- programs/portal/src/instructions/transfer_extension.rs
- programs/portal/src/lib.rs
- programs/portal/src/messages.rs
- programs/portal/src/ntt_messages/bpf_loader_upgradeable.rs
- programs/portal/src/ntt_messages/chain_id.rs
- programs/portal/src/ntt_messages/errors.rs
- programs/portal/src/ntt_messages/mod.rs
- programs/portal/src/ntt_messages/mode.rs
- programs/portal/src/ntt_messages/ntt_manager.rs
- programs/portal/src/ntt_messages/transceiver.rs
- programs/portal/src/ntt_messages/trimmed_amount.rs
- programs/portal/src/ntt_messages/wormhole.rs
- programs/portal/src/payloads/mod.rs
- programs/portal/src/payloads/token_transfer.rs
- programs/portal/src/queue/inbox.rs

- `programs/portal/src/queue/outbox.rs`
- `programs/portal/src/spl_multisig.rs`
- `programs/portal/src/transceivers/wormhole/accounts.rs`
- `programs/portal/src/transceivers/wormhole/instructions/admin.rs`
- `programs/portal/src/transceivers/wormhole/instructions/broadcast_id.rs`
- `programs/portal/src/transceivers/wormhole/instructions/broadcast_peer.rs`
- `programs/portal/src/transceivers/wormhole/instructions/receive_message.rs`
- `programs/portal/src/transceivers/wormhole/instructions/release_outbound.rs`

Out-of-Scope: Third party dependencies and economic attacks.

REMEDATION COMMIT ID:



- [5339724](#)
- [06ffd41](#)
- [9a47dc5](#)
- [d73c338](#)
- [f36cd03](#)
- [1fea60a](#)

Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW



SECURITY ANALYSIS	RISK LEVEL	REMEDATION DATE
INBOXITEM RELEASE INSTRUCTION IS NOT ENFORCED	HIGH	SOLVED - 09/08/2025
INSUFFICIENT M-TOKEN MINT VALIDATION DURING EARN PROGRAM INITIALIZATION	LOW	SOLVED - 09/08/2025

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
RISK OF YIELD OVER-DISTRIBUTION	LOW	RISK ACCEPTED
TOKEN 2022 PROGRAM IS NOT ENFORCED	INFORMATIONAL	SOLVED - 09/16/2025
MISSING FIELD IN INITIALIZER OF EXTGLOBALV2 ACCOUNT	INFORMATIONAL	SOLVED - 08/11/2025
RISK OF DOS DUE TO INSUFFICIENT ACCOUNTS VALIDATION	INFORMATIONAL	SOLVED - 09/16/2025
RISK OF UNHANDLED INDEX OUT OF BOUNDS ERROR	INFORMATIONAL	SOLVED - 09/08/2025

7. FINDINGS & TECH DETAILS

7.1 INBOXITEM RELEASE INSTRUCTION IS NOT ENFORCED

// HIGH

Description

The program provides two release instructions, `release_inbound_mint` and `release_inbound_mint_extension`, which are used to mint and transfer tokens to the recipient. The intended destination mint (either M-token or Extension-token) is encoded in the `InboxItem` and should determine which release instruction must be executed.

However, the program does not enforce the correct mapping between the `InboxItem` mint type and the corresponding release instruction. As a result, an `InboxItem` with an Extension-token destination can be incorrectly released using the M-token instruction. In this case, M-tokens are minted and locked into an intermediate portal token account, while the corresponding Extension-tokens are never minted to the recipient.

Because release instructions are permissionless, a malicious actor could deliberately release Extension-token `InboxItems` as M-tokens. This would result in other users' funds being locked without the corresponding Extension-tokens being issued, leading to a potential loss of assets for affected users.

[programs/portal/src/instructions/release_inbound.rs](#)

```
139 // Mint then transfer to ensure transfer hook is called
140 token_interface::mint_to(
141     CpiContext::new_with_signer(
142         ctx.accounts.token_program.to_account_info(),
143         token_interface::MintTo {
144             mint: ctx.accounts.mint.to_account_info(),
145             to: ctx.accounts.custody.to_account_info(),
146             authority: ctx.accounts.token_authority.to_account_info(),
147         },
148         token_authority_sig,
149     ),
150     principal,
151 )?;
152
153 onchain::invoke_transfer_checked(
154     &ctx.accounts.token_program.key(),
155     ctx.accounts.custody.to_account_info(),
156     ctx.accounts.mint.to_account_info(),
157     ctx.accounts.recipient.to_account_info(),
158     ctx.accounts.token_authority.to_account_info(),
159     ctx.remaining_accounts,
160     principal,
161     ctx.accounts.mint.decimals,
162     token_authority_sig,
163 )?;
```

```
202 pub fn get_inbox_recipient_token_account(
203     recipient: &Pubkey,
204     destination_mint: &Pubkey,
205
```

```

206     amount: u64,
207     token_authority: &Pubkey,
208     m_mint: &Pubkey,
209     token_program: &Pubkey,
210 ) -> Option<Pubkey> {
211     // Only bridging data
212     if amount == 0 {
213         return None;
214     }
215
216     // Bridging to extension, require intermediate portal token account
217     if !destination_mint.eq(m_mint) {
218         return Some(get_associated_token_address_with_program_id(
219             token_authority,
220             m_mint,
221             token_program,
222         ));
223     }
224
225     // Bridging $M, require user token account
226     Some(get_associated_token_address_with_program_id(
227         &recipient,
228         m_mint,
229         token_program,
230     ))
231 }

```

Proof of Concept

1. Redeem an **InboxItem** to receive Extension tokens.
2. Use the **release_inbound_mint** instruction to release the **InboxItem**.

```

1  it('extension tokens redeemed as M-tokens', async () => {
2      const { address: extAta } = await spl.getOrCreateAssociatedTokenAccount(
3          connection,
4          payer,
5          extMint.publicKey,
6          payer.publicKey,
7          true,
8          undefined,
9          undefined,
10         TOKEN_PROGRAM,
11     );
12
13     // check ext balance
14     const extTokenAccountInfoBefore = await connection.getAccountInfo(extAta);
15     const extParsedTokenAccountBefore = spl.unpackAccount(extAta, extTokenAccountInfoBefore, TOKE
16     const extAmountBefore = extParsedTokenAccountBefore.amount;
17
18     const getRedeemTxns = redeem(
19         additionalRedeemAccounts(mint.publicKey, extMint.publicKey, extAta),
20         undefined,
21         false, // this flag was set to false to redeem as M-token
22     );
23
24     const txIds = await ssw(ctx, getRedeemTxns(), signer);
25     let logs = await fetchTransactionLogs(provider, txIds[txIds.length - 2].txid);
26     console.log(logs);
27     logs = await fetchTransactionLogs(provider, txIds[txIds.length - 1].txid);
28     console.log(logs);
29     // bridge event log exists
30     expect(logs[15].startsWith('Program data: bEUUGiR+')).toBeTruthy();
31     expect(logs).toContain('Program log: Index update: 1000000000001 | root update: false');
32
33     // verify data was propagated (scaled-ui multiplier was updated)
34     const mult = await getScaledUIMult(connection, mint.publicKey);
35     expect(mult).toBe(1.000000000001);
36 }

```


Recommendation

To address this issue, it is recommended to enforce an **InboxItem** to be released by the correct instruction based on the destination mint.

Remediation Comment

SOLVED: The **M0 team** resolved this issue by validating the recipient's address for M0 tokens. This ensures that the **release_inbound_mint** instruction can only be invoked when:

- the recipient is the **intermediate portal token account** (for extension bridging), or
- the recipient is an **active M0 token holder** (for direct bridging to M0 tokens).

Remediation Hash

<https://github.com/m0-foundation/solana-m/commit/5339724dd3b0aa07339927b420ea2d7396f7a6b7>

7.2 INSUFFICIENT M-TOKEN MINT VALIDATION DURING EARN PROGRAM INITIALIZATION

// LOW

Description

The **Earn program** provides two variants of the `initialize` instruction:

- one intended for migration from a previous program version, and
- one for new deployments on other compatible SVM chains.

The migration variant, however, performs incomplete validation of the new M-token mint and does not fully compare it with the old mint's settings. Specifically:

- The new M-token mint uses the `ScaledUiAmount` mint extension, but the `new_multiplier` value is not validated.
 - If the new mint uses a higher multiplier than the one saved in the old global config account, it could result in **unexpected behavior** and **temporary inflation of the underlying token value**.
- The instruction does not validate that the **mint decimals** and **token supply** of the new mint match the old mint.
 - Migrating to a mint with different decimals may compromise the **economic integrity** of the protocol.
 - Failing to verify the supply introduces a risk that the new M-tokens were not correctly minted to all extension vaults and holders or the supply was inflated.

For new deployments on other chains, the program also does not enforce that the mint decimals of the new M-token are consistent with the decimals used on existing chains, which can lead to cross-chain inconsistencies and bridging errors.

These missing validations open the possibility of inconsistencies, potentially leading to **incorrect token valuations, bridging errors, or user fund misallocation**.

[programs/earn/src/instructions/admin/initialize.rs](#)

```
112 // Validate the m_mint has the correct extensions and that the global account has been given the
113 let extensions = mint_ext_data.get_extension_types()?;
114 let global_key = &self.global_account.key();
115
116 // 1. Must have the ScaledUiAmount extension and global account must be the authority
117 if !extensions.contains(&ExtensionType::ScaledUiAmount) {
118     return err!(EarnError::InvalidMint);
119 }
120
121 let scaled_ui_config = mint_ext_data.get_extension::<ScaledUiAmountConfig>()?;
122 if scaled_ui_config.authority != OptionalNonZeroPubkey(*global_key) {
123     return err!(EarnError::InvalidMint);
124 }
```

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:H/D:M/Y:M (2.0)

Recommendation

To address this finding, it is recommended to ensure that the new M-token mint is properly validated. Specifically:

- The `ScaledUiAmount.new_multiplier` must be **less than or equal to** the old multiplier.
- The **decimals** must remain unchanged during migration and new deployments.
- The **supply** of the new mint during migration must be **equal to or less than** the previous supply to prevent inflation and unbacked tokens.

Remediation Comment

SOLVED: The **M0 team** resolved this finding by validating the new provided mint and:

- Ensuring the mint has the correct amount of decimals
- Ensuring the new mint's multiplier is not greater than the old multiplier
- Ensuring the new token supply is equal or less than the previous supply

Remediation Hash

<https://github.com/m0-foundation/solana-m/commit/06ffd411db2982b92ab943707aae323dd4ff96e6>

7.3 RISK OF YIELD OVER-DISTRIBUTION

// LOW

Description

The `sync` instruction of the `m_ext` program's crank variant updates the last extension and `m` indices. It was adapted to handle scenarios where an extension is temporarily removed as an earner (i.e., not accruing yield) and later re-added.

However, the instruction assumes that the last `m` index is synchronized **before** re-adding the extension as an earner and syncing the last extension index again.

If this step is skipped, the program may calculate the new extension index incorrectly, potentially resulting in an over-distribution of yield.

[programs/m_ext/src/instructions/crank/earn_authority/sync.rs](#)

```
51 pub fn handler(ctx: Context<Self>) -> Result<()> {
52     // Convert the latest M multiplier to a u64 index
53     let scaled_ui_config = get_scaled_ui_config(&ctx.accounts.m_mint)?;
54     let current_multiplier: f64 = scaled_ui_config.new_multiplier.into();
55     let timestamp: i64 = scaled_ui_config.new_multiplier_effective_timestamp.into();
56     let current_index: u64 = multiplier_to_index(current_multiplier)?;
57
58     // If the vault M token account is approved as an earner, calculate the new EXT index and up
59     if ctx.accounts.vault_m_token_account.state == AccountState::Initialized {
60         // Calculate the new EXT index based on the last synced EXT index and the change in M ind
61         let last_ext_index = ctx.accounts.global_account.yield_config.last_ext_index;
62         let last_m_index = ctx.accounts.global_account.yield_config.last_m_index;
63
64         let new_ext_index = (last_ext_index as u128)
65             .checked_mul(current_index as u128)
66             .ok_or(ExtError::MathOverflow)?
67             .checked_div(last_m_index as u128)
68             .ok_or(ExtError::MathUnderflow)? as u64;
69
70         ctx.accounts.global_account.yield_config.last_ext_index = new_ext_index;
71     }
72
73     // Update the M index and timestamp to the current values on the M mint
74     ctx.accounts.global_account.yield_config.last_m_index = current_index;
75     ctx.accounts.global_account.yield_config.timestamp = timestamp as u64;
76
77     emit!(SyncIndexUpdate {
78         m_index: ctx.accounts.global_account.yield_config.last_m_index,
79         ext_index: ctx.accounts.global_account.yield_config.last_ext_index,
80         ts: ctx.accounts.global_account.yield_config.timestamp,
81     });
82
83     Ok(())
84 }
```

Recommendation

To address this issue, it is recommended to ensure that the last  index is fully synchronized **before** re-adding the extension as an earner and syncing the extension index.

Remediation Comment

RISK ACCEPTED: The **M0 team** accepted this risk and stated that operational measures will be implemented to ensure the extension index is properly synchronized before re-adding an extension as an earner.

7.4 TOKEN 2022 PROGRAM IS NOT ENFORCED

// INFORMATIONAL

Description

The `initialize` instruction of the `Portal` program allows an authority to initialize the program and configure the corresponding MO token mint.

Currently, the program permits setting a mint owned by either the legacy Token program or the new Token-2022 program. However, the `Earn` program relies on the `ScaledUIAmount` token extension, which is only available in Token-2022.

Because the `Portal` program does not enforce that the mint is created under Token-2022, an administrator could accidentally set a mint governed by the legacy Token program. This would break compatibility with the `Earn` program, preventing it from using required token extensions. As a result, such misconfiguration would cause a denial of service (DoS) of the platform, as tokens could not be processed as expected.

```
41 #[account(  
42     constraint = mint.mint_authority.unwrap() == token_authority.key() @ NTError::InvalidMintAut  
43 )]  
44 pub mint: Box<InterfaceAccount<'info, token_interface::Mint>>,  
45  
46 #[account(  
47     init,  
48     payer = payer,  
49     space = 8 + OutboxRateLimit::INIT_SPACE,  
50     seeds = [OutboxRateLimit::SEED_PREFIX],  
51     bump,  
52 )]  
53 pub rate_limit: Account<'info, OutboxRateLimit>,  
54  
55 #[account(  
56     seeds = [crate::TOKEN_AUTHORITY_SEED],  
57     bump,  
58 )]  
59 /// CHECK: [token_authority] is checked against the custody account and the [mint]'s mint_authori  
60 /// In any case, this function is used to set the Config and initialize the program so we  
61 /// assume the caller of this function will have total control over the program.  
62 ///  
63 /// TODO: Using UncheckedAccount here leads to "Access violation in stack frame ...".  
64 /// Could refactor code to use Box<_> to reduce stack size.  
65 pub token_authority: AccountInfo<'info>,  
66  
67 #[account(  
68     init_if_needed,  
69     payer = payer,  
70     associated_token::mint = mint,  
71     associated_token::authority = token_authority,  
72     associated_token::token_program = token_program,  
73 )]  
74 /// The custody account that holds tokens in locking mode and temporarily  
75 /// holds tokens in burning mode.  
76 /// CHECK: Use init_if_needed here to prevent a denial-of-service of the [initialize]  
77 /// function if the token account has already been created.  
78 pub custody: InterfaceAccount<'info, token_interface::TokenAccount>,  
79  
80 /// CHECK: checked to be the appropriate token program when initialising the  
81 /// associated token account for the given mint.  
82
```

```
82 | pub token_program: Interface<'info, token_interface::TokenInterface>,
83 | pub associated_token_program: Program<'info, AssociatedToken>,
84 | bpf_loader_upgradeable_program: Program<'info, BpfLoaderUpgradeable>,
85 |
86 | system_program: Program<'info, System>,
```

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:H/I:N/D:N/Y:N (1.5)

Recommendation

To address this issue, it is recommended to validate that the new M0 mint is owned by the Token 2022 program.

Remediation Comment

SOLVED: The **M0 team** resolved this finding by enforcing the use of the Token2022 program.

Remediation Hash

<https://github.com/m0-foundation/solana-m/commit/9a47dc50801f2c36e1b34499ea5d040d47d018a1>

7.5 MISSING FIELD IN INITIALIZER OF EXTGLOBALV2 ACCOUNT

// INFORMATIONAL

Description

The `m_ext` program's `migrate` instruction is designed to reallocate the old config account `ExtGlobal` into the new structure `ExtGlobalV2`. However, the new structure introduces a `pending_admin` field, which is not initialized during migration. As a result, the program compilation fails.

[programs/m_ext/src/instructions/migrate.rs](#)

```
186 // Create the new layout and
187 // update the m_mint and m_earn_global_account fields
188 let new_global = ExtGlobalV2 {
189     admin: old_global.admin,
190     ext_mint: old_global.ext_mint,
191     m_mint: ctx.accounts.new_m_mint.key(),
192     m_earn_global_account: ctx.accounts.m_earn_global_account.key(),
193     bump: old_global.bump,
194     m_vault_bump: old_global.m_vault_bump,
195     ext_mint_authority_bump: old_global.ext_mint_authority_bump,
196     yield_config,
197     wrap_authorities: old_global.wrap_authorities.clone(),
198 };
```

[programs/m_ext/src/state/mod.rs](#)

```
7 #[account]
8 pub struct ExtGlobalV2 {
9     pub admin: Pubkey, // can update config values
10    pub pending_admin: Option<Pubkey>, // pending admin for two-step admin transfer
11    pub ext_mint: Pubkey,
12    pub m_mint: Pubkey,
13    pub m_earn_global_account: Pubkey,
14    pub bump: u8,
15    pub m_vault_bump: u8,
16    pub ext_mint_authority_bump: u8,
17    pub yield_config: YieldConfig, // variant specific state
18    pub wrap_authorities: Vec<Pubkey>, // accounts permissioned to wrap/unwrap the ext_mint
19 }
```

BVSS

AO:S/AC:L/AX:L/R:F/S:U/C:N/A:C/I:N/D:N/Y:N (0.5)

Recommendation

To address this issue, it is recommended to populate the `pending_admin` field and set it to `None` during the migration.

Remediation Comment

SOLVED: The **M0 team** resolved this finding by adding the missing field.

Remediation Hash

<https://github.com/m0-foundation/solana-m-extensions/commit/d73c338026dcd823066fc83a0695be8a6e8e00de>

7.6 RISK OF DOS DUE TO INSUFFICIENT ACCOUNTS VALIDATION

// INFORMATIONAL

Description

The `set_mint` instruction in the portal program allows an administrator to configure a new M-mint and its custody account. However, it does not verify that the mint's authority matches the expected PDA derived from the `TOKEN_AUTHORITY_SEED`.

In addition, both the `set_mint` and `initialize` instructions fail to check whether the custody account is frozen.

As a result, setting a mint with an incorrect authority would prevent the program from minting new tokens when releasing `InboxItems`, leading to a denial of service (DoS). Likewise, configuring a frozen custody account would cause a DoS when attempting to send or receive tokens through the bridge.

[programs/portal/src/instructions/admin.rs](#)

```
634 #[derive(Accounts)]
635 pub struct SetMint<'info> {
636     pub owner: Signer<'info>,
637
638     #[account(
639         mut,
640         has_one = owner,
641         constraint = config.paused @ NTTErrors::NotPaused,
642     )]
643     pub config: Account<'info, Config>,
644
645     pub mint: InterfaceAccount<'info, token_interface::Mint>,
646
647     /// CHECK: This account is validated by its seeds
648     #[account(
649         seeds = [TOKEN_AUTHORITY_SEED],
650         bump,
651     )]
652     pub token_authority: UncheckedAccount<'info>,
653
654     #[account(
655         associated_token::mint = mint,
656         associated_token::authority = token_authority,
657         associated_token::token_program = Token2022::id(),
658     )]
659     pub custody: InterfaceAccount<'info, token_interface::TokenAccount>,
660 }
661
662 pub fn set_mint(ctx: Context<SetMint>) -> Result<()> {
663     ctx.accounts.config.mint = ctx.accounts.mint.key();
664     ctx.accounts.config.custody = ctx.accounts.custody.key();
665     Ok(())
666 }
```

[programs/portal/src/instructions/initialize.rs](#)

```

67  #[account(
68      init_if_needed,
69      payer = payer,
70      associated_token::mint = mint,
71      associated_token::authority = token_authority,
72      associated_token::token_program = token_program,
73  )]
74  /// The custody account that holds tokens in locking mode and temporarily
75  /// holds tokens in burning mode.
76  /// CHECK: Use init_if_needed here to prevent a denial-of-service of the [initialize]
77  /// function if the token account has already been created.
78  pub custody: InterfaceAccount<'info', token_interface::TokenAccount>,

```

BVSS

AO:S/AC:L/AX:L/R:F/S:U/C:N/A:C/I:N/D:N/Y:N (0.5)

Recommendation

To address this issue, it is recommended to enforce the new M-mint mint authority to be the correct PDA and to validate that the custody account is **Initialized**.

Remediation Comment

SOLVED: The **M0 team** resolved this finding by enforcing the new M-mint mint authority to be the correct PDA and validating that the custody account is **Initialized**.

Remediation Hash

<https://github.com/m0-foundation/solana-m/commit/f36cd039ce340d9523133333a1134904d8f230c6>

7.7 RISK OF UNHANDLED INDEX OUT OF BOUNDS ERROR

// INFORMATIONAL

Description

The `resolve_execute_vaa_v1` instruction is a read-only instruction that helps relayers determine how to redeem messages. It assumes that at least one whitelisted extension will always be available to serve as a fallback if the intended destination mint cannot be found. However, the program does not enforce a minimum number of whitelisted extensions. As a result, it is possible for none to exist. In such cases, the `resolve_execute_vaa_v1` instruction fails with an unhandled *index out of bounds* error.

Although this issue does not pose a direct security risk, it is recommended to handle such errors explicitly in order to improve program robustness and usability.

[programs/portal/src/instructions/resolve_execute.rs](#)

```
410 let ext_program = swap_global_data
411     .whitelisted_extensions
412     .iter()
413     .find(|ext| ext.mint.eq(&destination_mint))
414     .unwrap_or_else(|| {
415         // If the extension program is not found, fallback to first whitelisted extension
416         let fallback = &swap_global_data.whitelisted_extensions[0];
417         msg!(
418             "Extension for {} not found, falling back to first whitelisted extension: {}",
419             destination_mint.to_string(),
420             fallback.mint.to_string(),
421         );
422         fallback
423     });
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

To address this issue, it is recommended to ensure that the program verifies the presence of at least one whitelisted extension and returns a clear error if none are available.

Remediation Comment

SOLVED: The **M0 team** resolved this finding by explicitly handling the potential panic and returning an appropriate error message.

Remediation Hash

<https://github.com/m0-foundation/solana-m/commit/1fea60a0a4f3e9dd0e5ea9f8c08c23eeb5b72375>

8. AUTOMATED TESTING

Static Analysis Report

Description

Halborn used automated security scanners to assist with detection of well-known security issues and vulnerabilities. Among the tools used was `cargo audit`, a security scanner for vulnerabilities reported to the RustSec Advisory Database. All vulnerabilities published in `https://crates.io` are stored in a repository named The RustSec Advisory Database. `cargo audit` is a human-readable version of the advisory database which performs a scanning on Cargo.lock. Security Detections are only in scope. All vulnerabilities shown here were already disclosed in the above report. However, to better assist the developers maintaining this code, the auditors are including the output with the dependencies tree, and this is included in the cargo audit output to better know the dependencies affected by unmaintained and vulnerable crates.

Cargo Audit Results

ID	CRATE	DESCRIPTION
RUSTSEC-2024-0344	curve25519-dalek	Timing variability in <code>curve25519-dalek</code> 's <code>Scalar29::sub</code> / <code>Scalar52::sub</code>
RUSTSEC-2022-0093	ed25519-dalek	Double Public Key Signing Function Oracle Attack on <code>ed25519-dalek</code>

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.